Data Structures Practice Assignment

Comprehensive Solved Solutions for Questions 1, 2, 3, 4, and 8

Q1. What is a Data Structure? Give 3 real-life examples.

A **Data Structure** is a specialized format for organizing, processing, retrieving, and storing data in a computer's memory or storage so that operations can be performed efficiently. It defines the relationship between the data elements, the allowed operations on them, and the rules governing their access behaviors.

3 Real-Life Examples:

1. A Stack of Plates (Stack Data Structure)

In a cafeteria, clean plates are placed one on top of another. The last plate placed on the pile is always the first one to be taken off. This perfectly mimics the **Last-In, First-Out (LIFO)** mechanism used by computer stacks for function call logs and undo operations.

2. A Queue at a Ticket Counter (Queue Data Structure)

People standing in line to purchase movie tickets follow a strict order: whoever arrives first is served first. This maps directly to the **First-In, First-Out (FIFO)** mechanism used in printer task management and CPU request scheduling.

3. An Organizational Hierarchy Chart (Tree Data Structure)

A corporate structure charts a CEO at the top, followed by Vice Presidents, Directors, Managers, and individual employees. This hierarchical relationship represents a non-linear **Tree Data Structure**, exactly like file directories on an operating system.

Q2. Differentiate between Linear and Non-linear Data Structures.

Feature	Linear Data Structure	Non-linear Data Structure
Arrangement	Elements are arranged sequentially or linearly one after another.	Elements are arranged hierarchically or interconnected in a network.
Levels	Single level structure; every element has a unique predecessor and successor (except endpoints).	Multi-level structure; an element can attach to multiple elements.
Traversal	Can be traversed completely in a single pass/run.	Requires multiple passes or complex algorithms (like DFS/BFS) to traverse fully.
Memory Utilization	Often contiguous (like arrays) or predictable sequential links.	Typically non-contiguous; memory is utilized more dynamically and efficiently.
Time Complexity	Time complexity often increases linearly with size for simple searches.	Highly efficient for complex hierarchical operations and relationships.
Examples	Arrays, Linked Lists, Stacks, Queues.	Trees, Graphs, Heaps.

Q3. Explain the internal structure of an Array with a memory diagram.

An **Array** is a linear data structure containing a collection of homogeneous elements (elements of the same data type) stored in contiguous memory locations. Because the memory is contiguous, the exact address of any element can be directly computed using its index and the base address via the formula:

$$\textit{Address of } A[i] = \textit{Base Address} + (i \times \textit{Size of one element})$$

Key Attributes:

- **Base Address (BA):** The memory location of the first element (index 0).
- **Index:** A numerical offset representing the position of an element relative to the start (typically 0 to n-1).

Internal Contiguous Memory Diagram:

Below is a visual representation of an integer array containing 4 elements ***[10, 20, 30, 40]*** starting at Base Address 1000 (assuming 4 bytes per integer):

Index:	0	1	2	3
Value:	**10**	**20**	**30**	**40**
Address:	1000 (Base)	1004	1008	1012

Q4. Write the algorithm and C program for Array Insertion.

Algorithm for Inserting an Element at a Specific Position:

```
Given: An Array 'LA' with 'N' elements, 'ITEM' to insert, and target 'POS' (0-
indexed).

Step 1: Start
Step 2: Check if Array is full. If full, print "Overflow" and exit.
Step 3: Initialize counter J = N - 1
Step 4: Repeat Step 5 while J >= POS
Step 5:     Shift element right: LA[J + 1] = LA[J]
            Decrement J: J = J - 1
Step 6: Insert the new element: LA[POS] = ITEM
Step 7: Increment array size: N = N + 1
Step 8: Stop
```

C Program Implementation:

```
#include <stdio.h>

int main() {
    int arr[100] = {10, 20, 30, 40, 50}; // Initial array elements
    int n = 5;                           // Current size of array
    int item = 25;                       // Element to be inserted
    int pos = 2;                         // Index position where item is inserted
    int i;

    printf("Original Array: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("
");

    // Shift elements to the right to make space for the new item
    for(i = n - 1; i >= pos; i--) {
        arr[i + 1] = arr[i];
    }

    // Insert the element at target index position
    arr[pos] = item;
    n++; // Increment current size of array

    printf("Array after insertion: ");
    for(i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("
");

    return 0;
}
```

Q8. Explain Singly Linked List with a node diagram.

A **Singly Linked List** is a dynamic linear data structure where elements are not stored in contiguous memory locations. Instead, elements are connected sequentially via pointers. Each element in a linked list is known as a **Node**.

Structure of a Node:

A standard node consists of two distinct parts:

1. **Data Field:** Stores the actual value or payload of the element.
2. **Next Field (Pointer):** Stores the memory address of the succeeding node in the sequence. The last node's next field contains a `NULL` pointer, signifying the end of the list.

Visual Representation of a Singly Linked List:

The entire sequence is controlled via a special pointer named **Head**, which anchors the start of the list.